

AI-Powered Stolen Vehicle Recovery System

Presentation & Architecture Guide

This document provides a complete breakdown of the project to help you present it confidently. It explains the core purpose, the modern tech stack used, the artificial intelligence pipeline, and the real-time data flow.

1. Project Overview & Objective

The **Stolen Vehicle Recovery System** is a real-time, AI-driven surveillance and monitoring application. Its primary goal is to autonomously scan live traffic or CCTV camera feeds, detect vehicles, extract their license plates, and instantly cross-reference them with an authoritative RTO (Regional Transport Office) database.

It acts as an automated traffic enforcement and security portal that instantly flags:

- **Unregistered Vehicles:** License plates not found in the official database.
 - **Mismatched / Stolen Vehicles:** Vehicles where the physical attributes (Make, Model, Color) detected by the AI do not match the official registration records for that license plate (indicating fake plates or a stolen car).
-

2. The Technology Stack

This project utilizes a highly modern, decoupled architecture split between a high-performance Python AI engine and a sleek JavaScript web dashboard.

Frontend & Web Backend (The Dashboard)

- **Next.js (v16.2):** The core React framework used to build both the user interface and the API routes.
- **React (v19):** For building the component-based UI.
- **Framer Motion:** Powers the smooth, dynamic micro-animations and table loading effects to give the app a premium, high-tech "glassmorphism" feel.
- **CSS3 / Glassmorphism UI:** Custom styling utilizing blur filters, neon green gradients, and dark mode aesthetics to create a futuristic police/admin portal.

Database & ORM

- **Prisma ORM (v6.19):** A next-generation Object-Relational Mapper that safely connects the Next.js backend to the database.
- **SQLite:** A lightweight, highly responsive SQL database (`dev.db`) storing the official RTO vehicle records (Owner details, Make, Model, Color, Plate).

Artificial Intelligence & Computer Vision (The Engine)

- **Python (v3.11):** The backbone language for all machine learning inference.
- **PyTorch & Ultralytics (YOLO):** Deep learning frameworks powering the specific models (`vehicle_detector.pt`, `plate_detector.pt`) that draw bounding boxes around cars and plates.
- **PaddleOCR:** An incredibly powerful Optical Character Recognition engine tailored for reading license plates under challenging angles and lighting conditions.
- **OpenCV (cv2):** Used for video stream processing, frame cropping, and advanced morphological image enhancements (CLAHE, Tophat) before feeding frames to the OCR.
- **Flask:** A lightweight Python web server used to stream the processed MJPEG video feed directly into the Next.js dashboard.

External APIs & Scripting

- **DuckDuckGo Image Web Scraper:** A custom Python script that dynamically scrapes high-quality internet images of vehicles based precisely on their Make, Model, and Color (e.g., "Red Maruti Suzuki Swift") to populate the RTO Database locally without relying on generic placeholders.
 - **Twilio SDK:** Integrated into the backend for future expansion to send SMS alerts to authorities when a stolen car is tracked.
-

3. How the System Works (The Data Flow)

Here is the step-by-step lifecycle of how a vehicle is processed:

Step 1: Video Ingestion & Vehicle Detection

The Python engine (`main.py`) reads a video feed (e.g., `lulu2.mp4` representing a CCTV camera). It uses the **YOLO Vehicle Detector** to locate cars in the frame and crops out the specific vehicle.

Step 2: Attribute Classification & Plate Extraction

The cropped vehicle image is passed to specialized AI classifiers to guess the **Make/Model** and the **Color**. Simultaneously, the **YOLO Plate Detector** finds the exact pixel coordinates of the license plate on the bumper.

Step 3: Image Enhancement & OCR Reading

The tiny license plate crop is mathematically enhanced using OpenCV (Contrast Limited Adaptive Histogram Equalization - CLAHE) to make the text pop. **PaddleOCR** then reads the alphanumeric characters on the plate.

Step 4: API Handshake

The Python script bundles the data (e.g., *Plate: KL818731, Make: Tata, Color: grey*) and POSTs it to the Next.js API (`/api/live_data`). The script also streams the drawn video frames via Flask to `/video_feed`.

Step 5: Database Verification

The Next.js API receives the plate number and queries **Prisma/SQLite**.

- If the RTO record says the car should be a *Red Hyundai*, but the AI detected a *Grey Tata*, it flags a **MISMATCH (Stolen/Fake Plate)**.
- If it matches perfectly, it flags it as **VALID**.

Step 6: Real-time UI Update

The Admin Portal on the browser polls the API and dynamically updates the **Detected Vehicles** table. It uses Framer Motion to slide the new records into view. If a vehicle is spotted and is **VALID**, the portal seamlessly overrides the stock internet photo in the RTO Database tab and displays the **exact, real-time image cropped directly from the camera feed**.

4. Key Selling Points for Your Presentation

- **Fully Autonomous:** No human intervention is required; the AI flags suspicious vehicles instantly.
- **Premium User Experience:** The dashboard abandons boring legacy tables in favor of a responsive, cyberpunk-inspired Glassmorphism design.
- **Resilient AI Processing:** Employs image preprocessing (CLAHE) to ensure accurate OCR readings even in poor lighting or low-resolution CCTV footage.
- **Dynamic Image Syncing:** Features a custom scraping engine that downloads highly accurate reference images of cars to ensure the visual database is perfectly aligned with real-world paint colors and models.